Polymorphism

Polymorphism ("Many forms"): Allows client code to treat related objects as the same generic type, performing the correct task at runtime.

Relying on if/else statements or instanceof checks makes code verbose, error-prone, and a maintenance nightmare when adding new elements.

Implementation in 3 steps:

1. Create a common abstract class or interface
2. Implement and override (@Override) the methods in each concrete subclass.
3. Make client code depend on the superclass

- Cleaner, shorter client code and high extensibility (new subclasses can be added without modifying existing client code).

abstraction: Keeping only the relevant parts of an object while ignoring unnecessary details. Superclasses are more abstract because they contain fewer details.

In Java: Enables generic collections (e.g. ArrayList<FormAnimal>)

Overriding: Redefining an inherited method from a superclass
Overloading: Creating methods with the same name but different parameters in the same class.

Abstraction

From a process perspective, it is the act of bringing certain details to the forefront while suppressing all others. Its essence is preserving relevant information for a given context and forgetting irrelevant information (Jobn Guttag)

- As a verb (in programming): the creation of an interface or class definition that exposes necessary details for working with a componet while hiding others
- As a noun: A description or definition that leaves out unnecessary details, such as a class definition.
- The desing of an abstraction depends completely on its context

Two Common Problems with Abstraction.

- Leaky abstraccion: Other components end up relying on details not explicitly stated in the abstraction.
- Over abstraction: Too much information is forgotten or considered irrelevant, resulting in insufficient control or inadequate access for users.

Coherence Criteria for good Abstraction:
Meaningful labels and identifiers: Must be precise; class and variable names should be nouns or noun phrases, and method names should be verbs phrases.
context-aware labels and identifiers: Identifiers should be clear within the context where they are declared
Explicit declaration: All accessible features (public/protected elements, interfaces) and usage constraints must be explicitly declared or documented
Abstraction sufficiency: Must be complete, ensuring users can do everything they need to do with the objects of that class
Abstraction consisencess: Should not contain redundant mechanisms for the same thing unless required for compatibility or adapters.
Abstract classes and Interfaces in Java:
·Abstract classes (abstract class): Objects of an abstract class cannot be instantiated
·Abstract methods: Derived classes must implement these methods
Interfaces: Reusable definitions that classes implement using the implements Keyword.

Interface

·It is a contract that defines a set of methods (and constats) that a class must implement
·It specifies what a class must do, but not how to do it (leaving the implementation to the class)
· It is declared using the interface Keyword
Main Caracteristics
·No direct instantion: You cannot istantiate an interface directly (e.g. new MyInterface()) because it contains no implementation
· Multiple Inheritance of Behavior: Unlike classes, a class can implement multiple interfaces using the implements Keyword.
· Methods and variables: By default, all methods in an interfaces are implicity public and abstract, and all variables are public static final constants.
· Extensibility: An interface can inherit from one or more other interfaces using the extends Keyword